

# CPU Simulator - A Learning Tool

This document accompanies a Java project (CPU simulator) that can be used to learn-by-doing. However, you will learn about all kinds of CPUs and various types of assemblers, not just the project CPU. That project can be found at <https://github.com/Jim-Clack/CPU>. There is also a companion document to this one that has a broader focus: [Solution Architect Roadmap](#).

It means that the CPU simulator will be poorly designed, not fast, and not follow best-practices. Those things are for designing software, but we're simulating hardware.

For example, a hardware "bit" should be represented by a "bit" - one eighth of a byte. That would be a best-practice and would run fast. But this CPU simulator is designed to look like digital circuitry - hardware.

## ***1 - Building Blocks of a Computer***

Step 1 is to create the building blocks of the computer, a Bit, a Byte, and a LogicGate.

### **Bit**

We aren't worried about efficiency or performance, as we are trying only to simulate the way the hardware works. So we'll store a bit in the lowest bit of an integer. Keep in mind that you must ensure that it is always 0 or 1 when you set `val`. There are three ways of doing this, either (a) use the `&` and operator, or (b) use the `%` modulus/remainder operator, or (c) simply convert any non-zero value to a 1. Here is a look at the Bit class that you need to write. BTW - Each of these methods can easily be implemented in a single statement - one line of code.

```
public class Bit {
    private int val = 0;
    public Bit(Bit bit) ...
    public Bit(int val) ...
    public Bit(boolean val) ...
    public int getVal() ...
    public Bit inverse() ...
}
```

### **BByte**

We will use the term BByte instead of Byte because Java reserves the keyword Byte. IMPORTANT: The bits in a byte (or BByte) are numbered from right-to-left, with the least-significant bit being referred to as bit number zero. This is because of the numeric nature of a byte. When you are working with regular (decimal) numbers, the *units* position is on the right. Therefore, the binary *units* position in a byte is also on the right. Make sure you implement this correctly in `setBit()` and `getBit()` in your code.

| <u>Bit-Numbering Reference</u> |    |    |    |   |   |   |   |              |
|--------------------------------|----|----|----|---|---|---|---|--------------|
| 7                              | 6  | 5  | 4  | 3 | 2 | 1 | 0 | ⇐ Bit Number |
| 128                            | 64 | 32 | 16 | 8 | 4 | 2 | 1 | ⇐ Bit Weight |

```

public class BByte {
    public static int NumBits = 8;
    public static int MaxBitNum = 7;
    public static int MaxBitWgt = 128;
    public static int NextBitWgt = 256;

    private final Bit[] content = new Bit[NumBits];

    public BByte() ...
    public void set(BByte from) ...
    public void setBit(int bitNum, Bit bit) ...
    public Bit getBit(int bitNum) ...
    public void set(int from) ...
    public String toString() ...
}

```

*Hint:*

You'll need to make use of shift operators and logical (&) operators. And remember to create a new Bit each assignment, not just set the current bit to a new value. Why? Because, if you don't instantiate a new Bit, Java will allow multiple references to the same object - the same Bit - and if code were to change the value of one of them, it would change the other as well. For instance:

```

public void set(int from) {
    for (int i = 0; i < NumBits; i++) {
        setBit(i, new Bit((from & (1 << i)) > 0));
    }
}

```

## More BByte Methods

There is no need to implement the following BByte methods yet. But, as we move forward, you will have to add these as well, in order to support other needs. These will make use of bit movement (shift), bit-tests (0 versus 1), and logic (we will discuss LogicGates next).

Note that some methods will involve basic arithmetic, but the computer cannot (yet) perform arithmetic, so how could these be implemented in hardware? The answer is two-fold: (a) Those methods are only used for getting data into and out of the computer so they are not needed for basic logic and program flow, and (b) You will soon see how arithmetic can be implemented in hardware by combining LogicGates to create a HalfAdder and an Adder.

So, if you feel that you are ready to do so, go ahead and start implementing some of these BByte methods.

```

public BByte(String bits) ...
public BByte(int intVal) ...
public void set(BByte from) ...
public void set(String from) ...
public int getSignedValue() ...
public int getUnsignedValue() ...
public Bit isZero() ...
public BByte onesCompliment() ...
public Bit shiftLeft(int howFar) ...
public Bit shiftRight(int howFar) ...

```

## LogicGate

Since you are going to learn about logic gates and are not yet ready to implement them yourself, here is the entire source code for four kinds of LogicGates. Notice that they are implemented as a single enumeration. We will discuss these and make use of them as *the basic building blocks for any computer*.

```
public enum LogicGate {
    AND("0001"),
    OR("0111"),
    XOR("0110"),
    NAND("1110");

    private final Bit[] truthTable = new Bit[4];

    LogicGate(String truthTable) {
        for(int i = 0; i < truthTable.length(); i++) {
            this.truthTable[i] = new Bit(truthTable.charAt(i) % 2);
        }
    }

    public Bit gate(Bit a, Bit b) {
        return truthTable[(a.getVal() * 2) + b.getVal()];
    }

    @Override
    public String toString() {
        return this.name() + "[" + truthTable[0] + ", " +
            truthTable[1] + ", " + truthTable[2] + ", " + truthTable[3] + "]";
    }
}
```

## 2 - Understanding Basic Operation

The building blocks in the previous section are *everything* that comprises a computer except for I/O. You will soon see how these things can be combined in order to do addition, subtraction, multiplication, division, conditionals, loops, functions, and so forth. Once you understand what a Bit and Byte really are, and what the LogicGates do, you will be knowledgeable about digital electronics.

### Digital Electronics

A computer is built up from transistors. A transistor is like a valve or a faucet for electrons to flow through. Each transistor has an input, an output, and an emitter/gate that controls how much electricity can flow through it. Multiple transistors can be wired together in such a way to create a bit, a byte, or a logic gate. And that is all you need. Well, except for one minor detail: the clock.

In every computer there is a constant ticking of a clock, often referred to as an M1 cycle. When you mention your computer's clock speed, this is the clock you're referring to, not the time-of-day clock. In a computer there are many things going on at the same time, just like a multi-threaded application in software. And, similarly, there is a potential problem with synchronization. Imagine that you have to make a decision based on two inputs but they may not arrive at exactly the same time. As you might imagine, this is a problem.

So everything that occurs in the hardware is synchronized by the clock. The electronic engineers have designed the circuitry based on how long it takes for each event to occur and then determined the clock speed based on that. The way it works is that none of the LogicGates can gate - make a

decision to determine the output - until they get a clock signal. That signal is said to “latch the output”, which is to make the expected decision then “lock it in” so it doesn’t change until the next clock cycle.

Many of the LogicGates in the computer are implemented via flip-flops. A flip-flop is simply a gate that accepts a (clock) signal that will latch its output until it gets another signal. If the flip-flop is involved in program flow, then the M1 clock cycle is expected to latch it.

That is digital electronics in a nutshell. Soon, you’ll learn how we can combine these building blocks to create an entire computer.

## Logic Gates

You just saw how simple a logic gate is, as four of them were coded in a few lines of Java. A logic gate takes two binary inputs and, based on their state, generates a binary output state. For instance, with an AND gate, if two inputs are both at a state of 1 then the output of the gate is 1. But if either or both of the inputs are at a state of 0 then the output will be zero. That is why it is referred to as an “AND” gate - both input a AND input b must be at a 1 state in order to set the output state to 1.

*AND Gate: Outputs a 1 if  $a = 1$  AND  $B = 1$ , else it outputs a 0.*

For an OR gate, either input a OR input b must be at a state of 1 in order to set the output state to 1. If a is 1, or if b is 1, or if both are 1, then the output is 1. Hence, the name “OR” gate.

*OR Gate: Outputs a 1 if  $a = 1$  OR  $B = 1$ , else it outputs a 0.*

An XOR gate is referred to as an eXclusive OR gate because, like an OR gate, it outputs a 1 if a OR b are 1. But, unlike an OR gate, if both a and b are 1, it outputs a 0.

*XOR Gate: Outputs a 1 if  $a = 1$  OR  $B = 1$ , but not both a and b, else it outputs a 0.*

A NAND gate is identical to an AND gate except that the output is inverted.

*NAND Gate: Outputs a 0 if  $a = 1$  AND  $B = 1$ , else it outputs a 1.*

These are the four most common kinds of logic gate. They can be described with a Truth Table like this:

| a input | b input | AND output | OR output | XOR output | NAND output |
|---------|---------|------------|-----------|------------|-------------|
| 0       | 0       | 0          | 0         | 0          | 1           |
| 0       | 1       | 0          | 1         | 1          | 1           |
| 1       | 0       | 0          | 1         | 1          | 1           |
| 1       | 1       | 1          | 1         | 0          | 0           |

Everything beyond the simple wiring in a computer is performed by a variety of logic gates interconnected in various ways. Logic gates are responsible for arithmetic, program flow, function calls, and so forth.

## Unit Testing

Let's do some tests to verify that the building blocks work as expected. In order to do this, you may have to implement some of the additional BByte methods described in the previous section. Here's source code for LogicGateTest and BByteTest.

```
@Test
void testLogicGates() {
    testGate(LogicGate.XOR, (a, b) -> a ^ b);
    testGate(LogicGate.AND, (a, b) -> a & b);
    testGate(LogicGate.OR, (a, b) -> a | b);
    testGate(LogicGate.NAND, (a, b) -> 1 - (a & b));
}

void testGate(LogicGate gate, IntBinaryOperator tester) {
    System.out.println("LogicGate Test: " + gate);
    for(int a = 0; a < 2; a++) {
        for (int b = 0; b < 2; b++) {
            Bit bitA = new Bit(a);
            Bit bitB = new Bit(b);
            Bit result = gate.gate(bitA, bitB);
            System.out.println(" a=" + a + ", b=" + b + ", ==>" + result);
            int expected = tester.applyAsInt(bitA.getVal(), bitB.getVal());
            assertEquals(expected, result.getVal());
        }
    }
}

@Test
void testBByte() {
    System.out.println("BByte Tests:");
    testSignedValue(86, new BByte("01010110"));
    testSignedValue(54, new BByte(0x36));
    testSignedValue(-107, new BByte(-107));
    testSignedValue(-108, new BByte(107).onesCompliment());
    testSignedValue( 107, new BByte(-108).onesCompliment());
    testSignedValue( -128, new BByte(128));
    testSignedValue(  -2, new BByte(254));
    testSignedValue(  -1, new BByte(255));
    testUnsignedValue(  0, new BByte(0));
    testUnsignedValue( 127, new BByte(127));
    testUnsignedValue( 128, new BByte(128));
    testUnsignedValue( 254, new BByte(254));
    testUnsignedValue( 255, new BByte(255));
    testUnsignedValue( 255, new BByte(-1));
}

void testSignedValue(int expected, BByte actual) {
    System.out.println(" " +
        expected + " ==?= " + actual.getSignedValue() + " ==> " + actual);
    assertEquals(expected, actual.getSignedValue());
}

void testUnsignedValue(int expected, BByte actual) {
    System.out.println(" " +
        expected + " ==?= " + actual.getUnsignedValue() + " ==> " + actual);
    assertEquals(expected, actual.getUnsignedValue());
}
```

Make sure that these unit tests succeed before going on.

## Topics for Discussion

We need to talk about these things first. Then we can build the computer simulator.

- Universal Turing machine and procedural/imperative (or OO) programming
- Lambda Calculus and functional programming
- Little endian (versus big endian) - byte order
- Logic, shift, move, test - bit operations only
- No math except on real-world conversions (strings, decimal values)
- Note: iterating over bits at a low level is really "selection" not addition
- Two's compliment notation (negative values, numeric invert b boolean invert)

### 3 - Arithmetic and Logic

Maybe it seems too simple to you, or maybe it's overwhelming and difficult to understand. If you are not comfortable with these building blocks, do not go on. Review, experiment, research, and ask questions, until you fully understand it all - before you continue.

## HalfAdder

This is another kind of logic gate that comprises two logic gates. It is called a HalfAdder because you will need two of them to add two bits together. For instance, if we need to add two bytes, eight bits each, we'd need 16 HalfAdders. In hardware, this is exactly how it's done. But in software it is easier to just use two HalfAdders per bit and then iterate over the eight bits in the bytes.

Here is the logic for a HalfAdder:

```
sum = LogicGate.XOR.gate(p, q);  
carry = LogicGate.AND.gate(p, q);
```

ADD: Carry Flag = the value that gets discarded from the previous ADD or ADC  
ADC: Carry Flag = a value that gets carried to the units (rightmost) digit

```
10001010101001111010101010011111110100101010000010100100100100010111
11110100101010101000000010101001001000000000000000000000111111110001
```

1. ADD -----

## 2. ADC -----

### 3. ADC -----

1100

1010

— — — —

0 = sum, 0 = carry

1 = sum, 0 = carry

1 = sum, 0 = carry

0 = sum, 1 = carry

That's all there is to it. Notice that the sum is only one Bit. If we add two Bits, they might sum up to more than one Bit, so how is that handled? That is the purpose of the carry Bit. When we stack our HalfAdders to add up Bytes instead of Bits, we will have to handle the carry when we get to the next

bit to the left. We always add the bits from right-to-left. This is exactly like regular (decimal) addition and should be familiar to you. Please make sure you understand this:

How to “carry” the overflow from one bit/digit to the next one to the left:

|          |           |          |            |                |
|----------|-----------|----------|------------|----------------|
|          | <b>11</b> |          | <b>1 1</b> | ← <i>carry</i> |
| Decimal  | 168       | Binary   | 10101000   |                |
| Addition | + 45      | Addition | + 00101101 |                |
|          | -----     |          | -----      |                |
|          | 213       |          | 11010101   |                |

## Adder

How do we use 2 HalfAdders to make a full Adder?

- First of all, an Adder handles the carry to the next bit/digit, so it will always involve more than one Bit. Therefore, we will build our Adder by extending a BByte. The BByte that is used by the Adder is often referred to as an accumulator.
- Secondly, we'll have to write code that iterates over the bits, beginning with the rightmost bit, Bit 0, then incrementing to the left. The addition of each bit will involve two HalfAdders, one to add the bits and one to add in the carry. The first bit, the rightmost bit, Bit 0, will start with an incoming carry of 0, of course, because there are no bits to its right.
- Finally, we have to remember to keep track of the carry from each iteration so we can add it into the next iteration - the next Bit to the left. And remember: The right-most bit is Bit number 0.

Once you understand these three principles, you should be able to see how an Adder works:

```

1 ← carry
1011
0011
----
0 = sum, 1 = carry, A1
0 = sum, 0 = carry, A2

    carry = new Bit(0);
    for(int bitNum = 0; bitNum < BByte.NumBits; bitNum++) {
(A1)    halfAdder.add(this.getBit(bitNum), bbyte.getBit(bitNum)); // add bits
        Bit step1Carry = halfAdder.getCarry();
(A2)    halfAdder.add(halfAdder.getSum(), carry); // add carry
        carry = LogicGate.OR.gate(halfAdder.getCarry(), step1Carry);
        this.setBit(bitNum, halfAdder.getSum());
    }

```

That's it. This was the most complicated part of digital electronics. You now know everything you need to know to create an entire computer. Most likely, though, you do not yet know how to put all the pieces together.

See if you can write the code for the HalfAdder and the Adder. Here are the skeletons for you to fill out.

```

public class HalfAdder {
    private Bit sum = new Bit(0);
    private Bit carry = new Bit(0);
    public void add(Bit p, Bit q) {
        ... // ← You were told (see HalfAdder, above) what goes here
    }
    public Bit getSum() { return sum; }
    public Bit getCarry() { return carry; }
}

public class Adder extends BByte {
    protected Bit carry = new Bit(0);
    public BByte add(BByte bbyte) {
        HalfAdder halfAdder = new HalfAdder();
        carry = new Bit(0);
        ... // ← You were told (see Adder, above) what goes here
        carry = halfAdder.getCarry();
        return this;
    }
}

```

You can check your work against the HalfAdder.java and Adder.java in the accompanying CPU project.

## More Complex Operations

- Transistors in most computers are CMOS circuits built on a semiconductor Substrate.
- LogicGates are actually built from those Transistors.
- FlipFlops are built by combining LogicGates.
- Memory, HalfAdders, and Adders are made up from FlipFlops and LogicGates.

As you can see, we now have the ability to move data, Bytes and Bits, to perform Logical operations on it, and to do Addition on positive integers. What else is there?

- Handle negative integers?
- Perform subtraction?
- Compare for equality, less-than, and greater-than?

Believe it or not, those are freebies. Your code already handles all of those things. Really. We will soon discuss two's-compliment arithmetic, which is just a way of using our existing code to do all three of those capabilities with minimal effort. Yes, but how about...

- Multiplication?
- Division?
- Floating point numbers?

Simple: Those aren't handled directly by the hardware. The computer has subroutines, software stored as firmware within the CPU called "microcode" that does long-division and handles floating point numbers the same way you do on paper, multiple steps with simple integer operations. So it can all be done with an Adder. Of course, some CPUs have additional gates set up to perform common functions that will be called by the microcode, but the Adder is the key.

## Two's-Compliment Arithmetic



Negative numbers, subtraction, and comparison can all be performed by the Adder that you just created. Let's begin with negative numbers.

We will first discuss this approach using regular decimal numbers - ten's-compliment notation. Imagine that we have only two digits so we can count from 0 to 99, but we need to count from -50 to +49. We need negative values as well as positive values. All we need to do is to give certain numbers an alias. We can still use 0 to 49 as our positive integers. But we will use the values 50 to 99 to represent the negative values -50 to -1. So 97 is the same as -3, and 95 is the same as -5, for instance. How does this work?

Let's try adding -2 and -3. These two values will be represented by 98 and 97, respectively. When we add 98 and 97, we get 195. Of course that is too large to fit into our two digits, so the leading 1 is lost and we end up with 95. Notice that the alias for -5 is indeed 95. It worked!

As you can see from this example, we can use the values from 0 to 99 to represent an unsigned value from 0 to 99 or a signed value from 0 to 49 and -50 to -1. Think of that. The same math works on signed or unsigned values. All we need to do is to cover negative values to their appropriate value when we input them or output them. The exact same math works either way.

#### Ten's-Compliment and Two's Compliment Notation

Analogy to Two's-Compliment 8-Bits, but using Ten's-Compliment 2-Digits instead:

- For unsigned values  $\Rightarrow$  the range is 0-99.
- For signed values  $\Rightarrow$  values  $\geq 50$  will be assumed to be negative.

#### Addition Example

| Unsigned         | Signed |
|------------------|--------|
| 97               | -3     |
| 98               | -2     |
| ---              | ---    |
| 95=Sum (Carry=1) | -5     |

  

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| 95 == -5 | 96 == -4 | 97 == -3 | 98 == -2 | 99 == -1 |
|----------|----------|----------|----------|----------|

Example...

```
set(95) -> 95 (95 is simply the value 95)
set(-5) -> 95 (-5 is represented by 95 for a signed value)
```

Also, note that we can subtract a value (95) from the maximum possible value (99) then add 1 to negate a value.

Now, let's negate a Two's-Compliment 8-Bit value. Similarly, we subtract it from the maximum value (255), which is the same as inverting all the bits, then adding 1:

```
95 = 01011111
inverted (one's compliment) => 10100000
+1 add one => 10100001
```

The result is 161 unsigned but is also -95 as a signed value.

Thus, we can set(), add(), subtract(), and otherwise process values without concern for whether they are positive or negative, signed or unsigned. The only difference between them is how we interpret them in the getters:

```
number.getUnsigned(); // if bit 7 == 1, it's at least 128
```

```
number.getSigned(); // if bit 7 == 1, it's negative
```

We can do the same thing with binary numbers, using Two's-Compliment notation instead of Ten's-Compliment.. For one byte, 8 bits, we can use it to hold unsigned (positive) values from 0 to 255, or signed (+/-) values from -128 to 127. Either way the values and the arithmetic are identical - the only difference is how we interpret the values. In other words, when we retrieve a value, we can interpret it as signed or unsigned - hence `getSignedValue()` and `getUnsignedValue()`.

## Negation

A benefit of two's-compliment notation is that a number can be negated very simply, by inverting all the bits then incrementing it. Inverting the bits, converting zeros to ones and ones to zeros, is referred to as one's-compliment or logical inversion.

Consider the binary value of 23:

```
00010111
```

Invert the bits:

```
11101000
```

Then increment the byte:

```
11101001
```

As expected, this represents -23 in two's-compliment notation. Notice that this same approach can be used to negate the -23 back to a positive 23.

Since we know how to negate a value, we can perform subtraction. We simply negate the value to be subtracted then add it.

## Floating Point

Of course, much of computing involves scientific or technological values with a decimal point. As you learned in third or fourth grade, you can perform floating point arithmetic using integer operations, as long as you follow certain steps - an algorithm. Similarly, a computer uses integer arithmetic and some highly sophisticated algorithms to perform floating point arithmetic. In small computers, this is done in software - code libraries. In larger computers, it is done in microcode, code that is embedded into the CPU. Advanced computers have certain steps from some of these algorithms implemented in the hardware.

There is a standard for floating point values, IEEE 754, that facilitates all of the necessary mathematical operations and supports various lengths and precisions of numbers. So all modern general purpose computers represent integers in two's-compliment form and floating point numbers in IEEE 754 form. Generally, you won't need to know how to read or write floating point values in binary. Just in case you want to know how, here's a Quick-Ref:

[How an IEEE 754 Floating-Point Number is Represented](#)

Notes: 1-Sign bit, Seven 1's are special cases, Exponent is in excess-128 notation.

Significand has an assumed leading 1, which is omitted from the bits.

```
Example:      S Exces128 FractionPrefixedAssume1
              3 32222222 22211111111110000000000
              1 09876543 21098765432109876543210
              if these ^^^^^^ 7 bits are 1's, it's a special case of One/NaN/Inf
```

Breakdown of a floating point value...

1. If the 7-V bits are set, it's a special case of One/Nan/Infin, else:
2. The leftmost bit becomes the negative sign (0=positive, 1=negative)
3. Take the next 8 bits, subtract 128 (10000000), and that is E (signed)
4. Take the remaining bits, prepend a "1", and that is the mantissa in binary

## Flag Bits

After each arithmetic or logical operation, flag bits are set. These are one-bit values that can be read by the program. Often they are kept in a register for easy access. There are always at least three flag bits:

- **Zero Flag** - This indicates that the result (accumulator) is all zeros. In other words, if the result of the operation is a zero byte, then the flag is set to one. This can be confusing to noobs, as one indicates zero, and zero indicates one. But it makes sense when you understand that "true" equals 1 and the flag indicates zero.
- **Carry/Overflow Flag** - This is set only if the previous operation tried to carry over past the left of the leftmost bit, or in certain operations, past the rightmost bit. In other words, it means that the operation overflowed, so a bit of data was lost, generally because there was an outstanding "carry" when the Adders finished.
- **Sign/Neg Flag** - This confirms that the leftmost bit of the result is a one. For signed two's-compliment values, this indicates that the result is negative. Refer back to the two's compliment topic and examples and you'll see that negative values always have the leftmost bit set.

## Comparison

Whether we are comparing integers, floating point numbers, strings, or anything else, the comparison is always numeric and reduced to an integer comparison. For instance, to compare strings to place them in alphabetic order, we compare the numeric values of each character.

Comparing values is the same as subtracting them from each other, discarding the result, then checking the flags, as follows:

- If the Zero Flag is set, then the values are equal. (Because the result of the subtraction was zero.)
- If the Carry/Overflow Flag is set, then the subtracted value is larger.
- If neither of those Flags are set, then the subtracted value is smaller.

Summary: Addition, subtraction, negation, and comparison are all performed by an Adder.

Since you have already been using the CPU project, I will no longer show complete source code. Instead, I will put relevant snippets in the corresponding topics. Here is code from Register.java...

Excerpts (incomplete) from register.java:

```
public class Register extends Adder {
    private final FlagRegister flagRegister;
    public Register(FlagRegister flagRegister) {
        this.flagRegister = flagRegister;
    }
    public void invert() {
        onesCompliment();
        super.add(new BByte(1));
        syncFlagRegister();
    }
    public void and(BByte bbyte) {
        for (int i = 0; i < BByte.NumBits; i++) {
            super.setBit(i, LogicGate.AND.gate(
                getBit(i), bbyte.getBit(i)));
        }
        syncFlagRegister();
    }
    public BByte add(int bbyte) {
        BByte result = super.add(new BByte(bbyte));
        syncFlagRegister();
        return result;
    }
    public Bit shiftLeft(int howFar) {
        carry = super.shiftLeft(howFar);
        syncFlagRegister();
        return carry;
    }
    public void syncFlagRegister() {
        if(flagRegister != null) {
            flagRegister.setFlags(this);
        }
    }
}
```

You may have noticed, in the inset box above, that the constructor accepts an argument that is a FlagRegister. This is a special kind of register that keeps track of the conditionals (zero, carry, sign) of the last arithmetic or logic operation on another Register. So when a Register performs a calculation it must send those conditions to the FlagRegister. Special-purpose system Registers are not intended to set those flags, so that constructor argument is passed as “null.” We will discuss those Registers next and then, in the next section, discuss flags and conditionals.

## Registers

Whereas memory is simply an array of bytes (BBytes), in most cases, the registers are far more capable. Registers contain two’s-compliment values and are affiliated with Adders so that arithmetic and logic can be used with them. In rare cases, an Adder can be affiliated with regular memory, but it is typically implemented by first loading that memory cell into a register within the CPU. Registers are far faster than regular memory because they are inside of the CPU along with all the logic. But there are only a few registers.

On CISC computers, like the PC/MAC/Pentium, most registers are special-purpose. One is optimized as a counter, another is optimized for use as an indexer to point to elements of an array, and yet another is dedicated to use as a stack pointer. These registers have names that suit their purpose, for instance: SP = Stack Pointer, EIX = extended indexer, etc.

On RISC computers, such as a Smartphone/ARM, most registers are general-purpose and can be used as you wish, although certain registers are defaulted to certain purposes. These registers are most often numbered, like memory: register 1, register 2, and so forth.

### **Key Takeaways: (IMPORTANT)**

1. Integer arithmetic and logical operations need not be concerned with whether a value is negative or positive because the process is the same either way.
2. But an integer getter is where concern is necessary. There must be two getters, one for unsigned values and another for signed values. Each one processes the values differently. Most programming languages take care of this for you by requiring you to specify, in the declaration or cast, whether an expected value is signed or unsigned. In Java, the default is always "signed."

## **4 - Opcodes and Arguments**

The only significant functionality we have not yet discussed is program flow: conditionals, loops, Fdas

### **Opcodes**

On CISC computers, like the PC/MAC/Pentium, there are hundreds or thousands of opcodes, where many are just minor variations and some are specialized instructions for optimizing performance. On RISC computers, such as a Smartphone/ARM, there are usually only a hundred or fewer opcodes, which allows them to be shorter and for the compiled code to take up less memory and run faster. Each approach has its own benefits and drawbacks.

Here are some typical opcode mnemonics:

- LOAD - Take a value from memory (or from another register) and put it into a register.
- STORE - Take a value from a register and put it into memory.
- PUSH - Put a value onto the stack, where multiple values can accumulate in LIFO order.
- POP - Remove a value from the stack and put it into a register or, possibly, elsewhere.
- ADD - Add a value (from memory or another register) to the specified register.
- JMP - Change in program flow, like a goto statement, that resumes execution elsewhere.
- CALL - Push the program counter and jump to a subroutine, function, or method.
- RET - Pop the program counter, returning from a subroutine, function, or method.

### **Flags and Conditionals**

Some operations are conditional. Almost every assembler allows the programmer to append condition mnemonic suffixes to certain instructions, generally the JUMP opcodes, to make them execute only if a certain condition has been met. These conditions are kept in a FlagsRegister and are typically only set by arithmetic or logical operations. In other words, if you LOAD, STORE, or POP

a register, the condition flags will not be adjusted. But if you do an ADD, CMP, or an XOR, then the conditions in the FLAGS register will be set according to the result of that operation.

For instance, if you want your code to jump to the Exit: label if the last ADD operation had a result of zero, you would code:

```
JZE Exit:
```

This means “Jump, if the result was zero or equal, to the Exit label.”

You may wonder why we say “zero or equal.” The answer is due to two’s-compliment arithmetic. Remember that subtraction (SUB) is a form of addition (ADD) and that a comparison (CMP) instruction is merely a subtraction that discards the result. Look at these three operations and consider how they deal with two’s-compliment numbers.

```
ADD 77 + (-77)    ...or...  
SUB 77 - 77       ...or...  
CMP 77, 77
```

These are all the same operation with different pre- or post-processing. In the case of ADD or SUB, the result is “Zero.” But in the case of CMP, the result is discarded, so the result is “Equal.” Therefore, “Zero” and “Equal” are the same flag - the ZE flag. Here are the three most commonly used flags:

|    |               |                             |
|----|---------------|-----------------------------|
| ZE | Zero or Equal | (ZERO flag is 1)            |
| GT | Greater Than  | (SIGN flag is 1)            |
| LT | Less Than     | (ZERO and SIGN flags are 0) |

And here are their inverses - opposite in meaning:

|     |                       |                             |
|-----|-----------------------|-----------------------------|
| NZE | Not Zero or Equal     | (ZERO flag is 0)            |
| LTE | Less Than or Equal    | (SIGN flag is 0)            |
| GTE | Greater Than or Equal | (ZERO and SIGN flags are 1) |

These are the six conditions, based on the flags, that you need to know in order to perform conditionals, loops, and other program flow control.

One aspect of these flags that often confuses newbies is that the Zero flag is set to 1 if the Register is zero, or set to 0 otherwise. In other words, a Flag of 1 indicates zero. This is because a boolean (logical) True is 1 and False is 0 - so a Zero Flag of 1 indicates True: the result was zero.

Typically there is also a carry (CY) flag that indicates that a bit was lost, that it fell out of the end of the register during the last arithmetic or logic operation. And there is usually a flag to indicate whether interrupts are enabled or not - a flag that may not be alterable by the programmer.

## Arguments and Address Modes

In the previous topic, you may have noticed that data can be moved from/to a variety of places, such as a register, a memory cell, a value in the code, or the stack. How does it know where? The answer is that this is specified in the coded instruction - the assembled opcode - machine language. For instance, there may be several variations of the ADD instruction, where one of them adds a value from memory and another one adds a value from another register. This is referred to as the “address mode.”

But different assemblers have different syntax, sometimes for the same computer. You need to refer to the documentation for each assembler in order to understand how this is done. With some assemblers, you have to add a suffix to the opcode to indicate the address mode. Other assemblers use punctuation on the arguments to do the same thing. For instance, here are examples on three different assemblers:

| Opcode Instruction              | Assembler A   | Assembler B   | Assembler C     |
|---------------------------------|---------------|---------------|-----------------|
| Register 1 + Memory[1234]       | ADDMEM 1,1234 | ADD #1,[1234] | ADD REG1,1234   |
| Register 1 + 1234               | ADDIMM 1,1234 | ADD #1,1234   | ADD REG1,'1234' |
| Register 1 + Register 2         | ADDREG 1,2    | ADD #1,#2     | ADD REG1,REG2   |
| Register 1 + Local Variable 3   | ADDFRA 1,3    | ADD #1,[BP+3] | ADD REG1,(FP+3) |
| Register 1 + Memory[Register 7] | ADDIND 1,7    | ADD #1,[#7]   | ADD REG1,(REG7) |

*(For the example project that accompanies the document, the first approach was used, merely because it is a “mini” assembler and this is the simplest to code.)*

#### Common Address Modes:

- Register - Argument is a register that contains the value.
- Immediate - Argument is a constant - the value.
- Memory/Direct - Argument is in a memory cell with the value.
- Register Indirect - Argument is a register that points to the memory cell with the value.
- Memory Indirect - Argument is a memory cell that points to a memory cell with the value.
- Indexed - Argument is an offset from an index register, to a memory cell with the value.
- Local/Frame - Argument is an offset into the frame, to the memory cell with the value.

So the address mode may be specified via a suffix on the opcode or by certain punctuation on the operands, depending on the assembler. In the examples above, note that the first argument is typically the destination. This is generally true for all instructions other than the STORE opcodes. However, some assemblers replace LOAD and STORE with a MOVE instruction and it handles both the load and store functionality by requiring that the destination is always the first argument. In other words, when the register is the second argument, it becomes a STORE instruction.

The project associated with this document (CPU) uses the first approach (Assembler A, above) so the following instruction is a valid mnemonic. Can you figure out what it does?

**JLTEIMM Loop:**

You may wonder why we used address mode IMM (Immediate) and not MEM. (Memory) The reason is that we want to jump to the label at that memory cell, which is a constant. If we were able to use “**JLTOMEM Loop:**” then we would be telling it to fetch the *integer value* from that memory cell and use it as a pointer, telling it where to jump to. The MEM address mode refers to the content (value) of that memory location, not the memory location itself. This is often a point of confusion for new assembly language programmers.

**IMPORTANT:** In high-level languages, the differences between IMM, MEM, and IND are represented in various ways. Languages like C and C++ assume that a variable is a value (similar to IMM) if it was declared as a const, or that it contains a value (similar to MEM) unless it has a “\*” before it or a “->” after it, in which case, it is a pointer. (similar to IND) In Java and C#, variables are generally by value (like MEM) unless they are objects, in which case they are references. (Like IND)

Note that there are some computers, referred to as “two address” machines, that allow you to apply an address mode to both arguments. But most computers only allow it on the last argument. But in most assemblers, you will need to load data into a register before you can store it somewhere else in memory.

I will no longer show source code in its entirety for the project files, as you can pull them from github. Here is what Opcode.java looks like. Note that only a few of the opcodes are listed here.

```
public enum Opcode {
    JMPIMM(14, 1),
    JZEIMM(15, 1),
    JGTIMM(16, 1),
    CALLIMM(30, 1),
    PUSHREG(31, 1),
    POPREG(32, 1),
    ADDIMM(111, 2),
    ADDREG(112, 2),
    ADDMEM(113, 2),
    ADDFRA(114, 2),
    ADDIND(115, 2),
    CMPIMM(135, 2),
    LOADREG(142, 2),
    STORMEM(151, 2);

    private final int value;
    private final int numArgs;
    private static final HashMap<Integer, Opcode> mapByValue = new HashMap<>();
    private static final HashMap<String, Opcode> mapByMnemonic = new HashMap<>();

    Opcode(int value, int numArgs) {
        this.value = value;
        this.numArgs = numArgs;
    }

    public static Opcode opcode(int val) {
        ensureMapsAreInitialized();
        Opcode opcode = mapByValue.get(val);
        if (opcode == null) {
            opcode = INVALID;
        }
        return opcode;
    }

    public static Opcode opcode(String mnemonic) {
        ensureMapsAreInitialized();
        Opcode opcode = mapByMnemonic.get(mnemonic.trim().toUpperCase());
        if (opcode == null) {
            opcode = INVALID;
        }
        return opcode;
    }

    public int getValue() {
        return this.value;
    }

    public String getMnemonic() {
        return this.name();
    }

    public int getNumArgs() {
        return numArgs;
    }
}
```



Here are source code excerpts showing how the opcodes are handled.

CPU.java

```
Opcode opcode = Opcode.opcode(getNextProgramByte());
if (opcode.getNumArgs() < 1) {
    runMode = execute(opcode);
} else {
    int argument1 = getNextProgramByte();
    if (opcode.getNumArgs() < 2) {
        runMode = execute(opcode, argument1);
    } else {
        int argument2 = getNextProgramByte();
        runMode = execute(opcode, argument1, argument2);
    }
}
```

Substrate.java

```
protected int getNextProgramByte() {
    int arg = memoryCells[registers[IP].getUnsignedValue()].getUnsignedValue();
    registers[IP].increment();
    return arg;
}
```

ExecuteTwoArgs.java

```
protected RunMode execute(Opcode opcode, int argument1, int argument2) {
    switch(opcode) {
        case ADDIMM:
            getRegister(argument1).add(argument2);
            break;
    }
    ...
}
```

Study the code above. Follow the flow and notice how it works:

- nextProgramByte() is reading the assembled code: opcodes and arguments
- Opcode.opcode() interprets and returns the opcode (sources on previous page)
- execute() simulates the CPU - performing an ADD immediate opcode in this case
- You should already be familiar with the BByte.getUnsignedValue() method

You are now knee-deep in the CPU project and should be able to pull, build, and run it. There are unit tests you can run. There is also a Main test program that loads a console driver. When you are running Main, you can type a number from 0-99 then hit Enter and the CPU simulator will input the value, process it by incrementing it, and then output it again. We will discuss the breakdown of these operations in an upcoming topic.

## Functions, Methods, Subroutines

As you have seen, program branching is simple, it is just a matter of loading a destination memory location into the IP (Instruction Pointer) register so that execution resumes from that point. That is the essence of a JMP opcode. And you've seen how the Flags can be used to implement conditions, for instance, to make a JMP occur only if certain conditions have been met. Putting those together,

you can see how “if,” “while,” and “for” instructions are performed in high-level programming languages.

But what about function calls, subroutines, methods, or procedures. These involve, not only a JMP to a different location in the code, but an ability to return to just after the JMP when the function is done. To understand the answer to this, you’ll first need to learn a bit about the stack. We will go into this more in forthcoming topics, but for now, here is a simple overview:

All modern CPUs are considered to be “register-based” as well as “stack-based.” In years passed, some CPUs were one or the other, but these days it’s both. You’ve seen that registers are simple memory cells that are “inside of” the CPU and typically affiliated with Adders. Two of those registers are used for keeping track of the Top of Stack (ToS) and the Stack Frame. These are the Stack Pointer (SP) and Frame Pointer. (FP)

You may already be familiar with the fact that all objects and member variables reside on the heap. Where are the others? They’re on the stack. Function arguments, local variables, and temporary values reside on the stack. As new memory is needed, the stack grows. As variables go out of scope, the stack shrinks. The Stack Pointer SP keeps track of the next available (yet unused) space in memory. The Frame Pointer FP keeps track of where the current function’s parameters and local variables are - somewhere within the stack.

**SP: Stack Pointer - Next available (unused) space for the stack to grow into.**  
**FP: Frame Pointer - Where parameters and locals for the current function reside.**

As your code creates temporary results and then consumes them, the stack grows and shrinks. When your code calls another function, the stack grows. When that function returns, the stack shrinks.

As I said above, parameters and locals for the currently-running function are all in the stack frame. But there is one more important variable on the stack - stored in the stack frame: The return address. That is a pointer to the location in your code that follows the call to the current function. In other words, when your function is called, the low level code has to push the arguments onto the stack as parameters and push the return address onto the stack as well - the address of the next opcode after the call.

How a CALL (to a function, method, or procedure) occurs at a low level:

- 1. The arguments to the method are pushed onto the stack to become parameters.*
- 2. The address of the next opcode after the call (return address) is pushed onto the stack.*
- 3. The call to the function is performed just like a JMP opcode with the addition of the previous step.*
- 4. The function that gets called preserves the registers by pushing them onto the stack.*
- 5. The function that gets called allocates space on the stack for locals and sets the Frame Pointer FP.*

...When the function is done, the following occurs at a low level...

- 6. The stack is discarded up to the frame, freeing up space taken by local variables and temporaries.*
- 7. The previous function’s state is restored by popping the registers back off of the stack.*
- 8. The instruction pointer is popped from the stack, resuming execution at the opcode after the call.*
- 9. The adjusts the stack, if necessary, discarding any arguments it passed.*

At a low level, the steps above can be performed individually or en-masse by certain opcodes. For instance, in the accompanying CPU project, there are opcodes to push and pop registers from the stack, to set, get, increment, or decrement the SP and FP registers, to JMP, and to return RET from a function. But there are also instructions that can be used in the function that perform multiple

steps. The Call (CALLxxx) opcode performs steps 2 and 3 above, the ENTER instruction handles steps 4 and 5, and the LEAVE opcode handles steps 6, 7, and 8.

In the companion document (Solution Architect Roadmap) there is a more in-depth explanation of the Stack Machine, with examples. This is an important concept in low-level coding.

## ***5 - Putting it All Together***

### **The Heap**

As just discussed, function parameters, local variables, and temporaries reside on the stack. But objects and their member variables reside on the heap. So do static variables and constants in Java, although in some programming languages, statics and constants reside in a separate fixed data area. So, what is the heap?

The “heap” is a term that refers to an area of memory where dynamic data resides - data that exists for a controllable period of time.

- DataSeg/BSS - An area where statics, constants, and globals reside for the duration of a run.
- Stack - An area where parameters and locals reside for the duration of a function call.
- Heap - An area where everything else resides, regardless of its lifetime.

We don't want memory leaks, as the program will crash if we run out of memory, so the heap has to be kept clean. Data that is no longer needed must be deleted from the heap. This is a much bigger problem than it first appears to be. Remember that memory alone does not have an “in-use” flag. We cannot just search for zeroed-out areas to find a slot in which to put an object. So if we just stuff data into the heap, there's no way to find empty areas to re-use. So there must be some organization to it. If we want to flag it as in-use, we need to subdivide it and put headers on each subdivision - headers to indicate size and whether it is in-use. As it turns out, over sixty years ago, this was the biggest problem in Computer Science and three solutions came out of that:

1. Manual Memory Management
2. Reference-Counting
3. Garbage Collection

All three of these approaches are typically based on this underlying principle:

Treat the unused area of memory as a linked-list. It starts out as one large slot that occupies the empty space and gets broken up as needed - subdivided into many nodes in that linked-list. When a node is no longer needed, it not only gets flagged as no longer in-use but is also merged with any adjacent nodes that are no longer in-use. When a new slot is needed, an exhaustive search for the best-fit turns out to be very inefficient, so we accept the first empty slot that it will fit into - hence the name First-Fit Heap, or simply Heap.

There are many factors that complicate memory management but the most confusing one is called “aliases.” As you know, when you call a method, one of the arguments might be an object that will be placed into a member variable of that object. Then that object might instantiate other objects, passing that same value to each of them. After a while, there could be five, ten, or a thousand variables that all reference that same object. Systems programmers refer to these as aliases. As you

might imagine, it would be bad to free up a Customer object when there is still an Order to be processed that references that Customer. We need to make sure that allocated areas of memory are kept alive until there are no more references, but no longer. So we need to know exactly when that last reference is gone.

*Manual Memory Management* is the way C, Pascal, and C++ deal with the heap. They expect the programmer to allocate and free memory, complicating his task. This introduces the concept of ownership, which code or data “owns” each allocated area of memory and is responsible for freeing it up when it’s no longer needed? This leads to many serious bugs and is error-prone. There are various implementations, such as C’s alloc/free, the C++ approach with new/delete, and the Pascal approach of mark/release, which sounds ideal but, in practice, is like programming while wearing boxing gloves. C++ has added support for smart pointers, a design pattern that greatly reduces the incidence of errors by wrapping pointers in self-managing objects. But the programmer still has memory management to deal with.

*Reference Counting* - Consider the prior approach, manual memory management, with the addition of one field at the beginning of each allocated area of memory: RefCount. Each time an alias is created, RefCount must be incremented. Each time an alias falls out of scope or gets freed, we need to decrement RefCount. That sounds easy enough, doesn’t it? But that means that every time you pass an object to a method, you have to worry about the other objects referenced by that one and deal with their RefCount. The same is true when you no longer need a variable. It gets really messy. Apple has a solution called Automated Reference Counting (ARC) in Objective-C and Swift that automates a lot of this, making use of both compile-time and run-time ARC management code.

*Garbage Collection* - By far the easiest and most thorough approach is Garbage Collection. However it is the slowest of the three, can be very inefficient with memory, and cause the program to have erratic brief pauses. But since computers have gotten so much faster and memory so much more available, this has become the preferred approach. Garbage Collection uses a Mark-and-Sweep approach, where it first performs “marking” a brute-force reference count on every object to map out the in-use areas, then it “sweeps” away the areas that are not in-use. Typically memory is divided into “Generations,” there are multiple heaps, one for objects created when the app first comes up, one for items that were newly created, and one for objects that have been around for a while. Of course, that means that in-use areas may be relocated, so the application code cannot use pointers, only handles - indexes into a table of pointers.

## Objects and “this”

Being a Java programmer, you might not be aware of how “this” works. When you code the following:

```
Class Test {
    int a = 0;
    void myMethod(int b) {
        ...
    }
}
...
int main(String args[]) {
    Test test = new Test();
    test.myMethod(7);
}
```

The compiler treats it as if you coded this instead:

```
Class Test {
    int a = 0;
    void myMethod(Test this, int b) {
        ...
    }
}
...
int main(String args[]) {
    Test test = new Test();
    test.myMethod(test, 7);
}
```

Java changes your method declarations so that they have an additional first parameter. Then whenever you call a method, the compiler inserts a first parameter “this.” You don’t need to compensate for this when you’re doing reflection because reflection is based on your declaration, not the generated code. But if you are working with instrumentation then you need to be aware of this.

In some languages, such as Rust, the compiler does not do this - you the programmer are expected to do this - to write your method declarations and method calls so they pass “self” explicitly. Python expects you to do this in method declarations but not in method calls.

But, as you can see, “this” is really not that special. It’s just the first parameter and it gets passed automatically.

## Assembly Language Reference

The associated project has an Assembler that translates opcodes and data into the machine code for this CPU simulator. Here is a quick-start reference for you.

### Meaning of opcode Mnemonic:

Operation[Condition][AddrMode]

### Condition:

|                    |                             |
|--------------------|-----------------------------|
| ZE = Zero or Equal | NZE = Not Zero or Equal     |
| LT = Less than     | LTE = Less Than or Equal    |
| GT = Greater than  | GTE = Greater Than or Equal |
| CY = Carry         | NCY = Not Carry             |

### AddrMode:

IMM = Immediate value - value is Arg2 literally  
REG = Register number - designated register Arg2  
IND = Indirect - memory as specified by the designated register Arg2  
MEM = Memory address - memory as specified by Arg2  
FRA = Stack frame offset from FP (only after ENTER opcode) see Note3  
Note1: for 2 Arg Ops: Arg1 is always REG so the AddrMode is for Arg2  
Note2: AddrMode for source, except STORxxx where it's destination  
Note3: FRA AddrMode 1, 2, 3... for params; 0, -1, -2... for locals

### Each line of assembly code contains a series of the following tokens:

[label:] [address:] [[opcode[,]] [arg[,]...] [label:] [byte[,]...] ["string"]

### Registers:

\$0 .. \$9 User registers - General purpose accumulators/indexers  
\$FLAG Flag register - Bit Weights: 1=Zero, 2=Carry, 4=Sign, 8=EnabIntr  
\$FP Frame Pointer - Maintained by ENTER/LEAVE/ILEAVE opcodes  
\$SP Stack Pointer - Used by PUSHxxx/POPxxx,ENTER/LEAVE/ILEAVE/RET  
\$IP Instruction Pointer - System Use ONLY  
\$IV Interrupt Vector - Set this to your ISR or ISR discriminator  
\$IN Interrupt Number - System sets this to the current IRQ, 0 = None  
\$SC Internal Scratch Register - Overwritten by arith/logic opcodes

### These can be newline-delimited or semicolon-delimited.

- If the address is omitted, it increments starting from zero.
- Opcodes are not case-sensitive.
- Spaces are optional and ignored.
- Comments begin with # and continue to the end of line or a semicolon.
- If a label is left-justified then it's a target instead of a reference.
- ConstantArgs/Bytes can start with "0x" if they are hex instead of decimal.
- Register numbers start with "0\$" or "\$", required if it's a register name.
- Math/logic operations on System registers do not set ZE/GT/LT flags.
- There is a directive/pseudo-op EQU for creating numeric constants as well.

### Calling Convention:

|                              |                                      |
|------------------------------|--------------------------------------|
| PUSHxxx params...            | (Push arguments onto stack)          |
| CALLIMM xxxx                 | (Call the function)                  |
| ADDIMM SP, ParamsSize        | (Done upon return)                   |
| Function Entry:              | (Or use the ENTER nnn opcode)        |
| PUSHREG \$0 ... PUSHREG \$FP |                                      |
| LOADREG \$FP, \$SP           |                                      |
| ADDIMM \$SP, nnn             | (Where nnn = LocalsSize)             |
| Function Exit:               | (Or use the LEAVE opcode, see Note1) |
| LOADREG \$SP, \$FP           |                                      |
| POPREG \$FP ... POPREG \$0   |                                      |
| RET                          | (See Note1)                          |

\*Note1: For an ISR, use ILEAVE/IRET instead of LEAVE/RET

### FP offsets, assuming there are 2 one-byte parameters and 3 one-byte locals:

|         |      |                                                |
|---------|------|------------------------------------------------|
| PARAM_2 | FP+2 |                                                |
| PARAM_1 | FP+1 | (xxxFRA Parameters have a positive offset)     |
| ...     |      |                                                |
| LOCAL_0 | FP   | (xxxFRA Locals have a zero or negative offset) |
| LOCAL_1 | FP-1 |                                                |
| LOCAL_2 | FP-2 |                                                |

## Assembly Language

The following example is derived from the unit tests for the CPU simulator project. This example simply demonstrates how to perform a function call, passing parameters and allocating space for local variables.

```
# byte sum = Func(1, 2);  
    LOADIMM $5, 1  
    LOADIMM $6, 2
```

```

        PUSHREG $5          # push Arg 1, val=1
        PUSHREG $6          # push Arg 2, val=2
        CALLIMM Func:
        POPREG $1
        POPREG $2
        STORMEM $2, Sum:    # save result to Sum:
        JMPIMM End:
Sum:    0                   # 24: result goes here

# byte Func(byte a, byte b) { return a+b; }
Func:   ENTER 1             # allocate 1 local Var
        LOADFRA $7, 2       # get Arg 2 into Reg 7
        ADDFRA $7, 1        # add Arg 1 into Reg 7
        STORFRA $7, 0       # store Sum in local Var
        STORFRA $7, 2       # store it to Arg 2
        LEAVE
End:

```

The first four lines push 2 arguments onto the stack. The next line calls the function “Func:”. Upon return, it has to pop two values off the stack to read and discard the parameters it pushed. When it does this, it also stores the value of Arg2 to “Sum:”.

Note that code and data are intermixed. On many assemblers the code and the data get stored into separate areas, but here we have to jump over the data: “JMPIMM End:” so that it doesn’t try to “execute” data.

Let’s look at the function itself, “Func:”. Upon entry, it allocates space for one local variable - a variable that is redundant, but is here to provide you with an example. Then it loads “Arg2:” into register \$7 then adds “Arg1:” to it. It stores the result in the local variable then into “Arg2:”. Finally, it leaves the function.

Notice that registers always begin with a “\$” and that labels always end with a “:”. Although this punctuation is optional in certain cases, it is a best-practice to always punctuate properly so that your code is readable.

If you look at `AssemblerTest.java`, you will see many more examples. The next topic also involves some assembly code.

## 6 - Input and Output

### I/O Ports

There are several ways that I/O devices communicate with the CPU:

- Memory Space - The adaptor just looks like memory cells to the CPU.
- I/O Space - The CPU has an alternate memory-like space just for I/O devices.
- Direct Memory Access - DMA - The adaptor reads from and writes to memory directly.

Often, more than one of these approaches is used by a single adaptor. For instance, it may use memory space or I/O space to communicate with the CPU about setup, status, and control, whereas data will be exchanged via DMA.



To use the first approach, *memory space*, the CPU reads the I/O adapter by using a LOAD opcode to read it just as if it's memory. And the CPU can send to the adapter using a STORE opcode as if it were writing to memory. The adapter will have a reserved area of memory that it appears in.

To use *I/O space*, the CPU has special opcodes, typically called IN and OUT or INPUT and OUTPUT. They act like LOAD and STORE opcodes except that they access I/O space where the adapter can be found.

A variety of means are used for *DMA*, entirely dependent on the adapter, when configuring it or exchanging status information. But the data will be read from or written to an area of memory that is selected by the CPU, which will need to send that location to the adapter when it configures it.

You may be wondering how a driver gets activated by the hardware. For example, when an email arrives over the network, the device reads it and sends it to the CPU via the adapter. But how does the adapter get the attention of the CPU? When you move the mouse, even a small amount, the CPU has to be informed so that it can move the cursor accordingly, but how does the CPU know that it needs to do this? If a scheduled event comes due, the real-time-clock has to get the CPU to notice it, but how? If an I/O device has sent data to the CPU by putting it into an I/O port, how does it tell the CPU that it should read that port? The Answer: These things are all managed by a combination of hardware and software using interrupts. That's the next topic.

## Interrupts

You now understand how software can talk to hardware. If you needed to send output to the console, you might have to send it to an I/O port or write it to an I/O-mapped area of memory. All is well and fine. When the hardware wants to talk to the computer, it can put data in an I/O port or write it to I/O-mapped memory, but that doesn't make the software read it. How does hardware get the attention of the CPU?

An interrupt is an electronic signal sent to the CPU, typically by some device or I/O adapter. Generally, the software that deals with the interrupt is a driver. This is exactly like a function call, except that it is typically initiated by hardware. This might trigger some alarms in your brain...

1. Does it interrupt my code right in the middle of its work?
2. How does it do this without causing chaos, corrupting the state of the CPU and registers?
3. How can it differentiate between different I/O devices to know which one interrupted it?
4. What if more interrupts occur while the CPU is already handling one?

These are all important questions. Here are the answers...

1. Yes, it interrupts your code. So it must do its work very quickly.
2. It has to preserve the registers and CPU state then restore it when it is done.
3. There is usually a way to send an interrupt number. Otherwise, it's trial-and-error.
4. Interrupts are disabled when it is already handling one, so they are handled in sequence.

In a typical desktop computer, hundreds of interrupts can be handled every second. As you move your mouse, the interrupts pour into the CPU so it can keep the cursor updated. When you are running many applications and some of them have multiple threads, the dispatcher is dealing with a flood of interrupts. Yes, wow!



Typically, what happens is that there is firmware on an internal device or an I/O adapter that sends the interrupts. Each interrupt triggers an ISR (Interrupt Service Routine, a special kind of function) to tell it to handle the I/O. That ISR has no API for the OS to call but, instead, simply pumps data through an input queue and/or an output/queue, then returns to let the computer continue where it left off. The ISR preserved and restored the registers and everything, so that programs don't crash.

Then another part of the driver, the API, reads or writes the queues and communicates with the operating system. If the I/O data pertains to your program, then your program will be calling the run-time libraries (net, stream, file, etc.) which, in turn, talk to the operating system. You might think that this is inefficient but, once you understand how it works you will appreciate that it is very streamline and secure.

## Device Drivers

In the associated CPU project, take a look at Main. It makes use of a console device driver named SampleIoDevice. That class contains several things all wrapped up in one file:

1. The code to instantiate a CPU simulator as well as an Assembler.
2. Wiring to connect the device to the CPU. (simulated, of course)
3. The console simulator itself, written in Java to do I/O with System.out and System.in.
4. A device driver in assembler consisting of the API and the ISR.

Here's some of the code and a summary of how it works:

Step 1 - Create a CPU simulator and an Assembler then assemble and load the device driver..

```
CPU cpu = new CPU();
new Assembler(cpu).assemble(driverSourceCode);
```

Step 2 - Wire it up. Connect to the output port, input port, and interrupt.

```
cpu.ioPorts[OutputPort].setDeviceCallback(this);
cpu.ioPorts[InputPort].setInterruptNumber(InputIRQ);
```

Step 3 - Here's the device simulator that connects the I/O ports to System.in and System.out.

```
Thread simulator = new Thread() {           // Keyboard input from System.in
    public void run() {
        System.out.print("Type 0..99 and hit Enter>");
        Scanner keyboard = new Scanner(System.in);
        while(true) {
            String command = keyboard.nextLine();
            if (!command.isEmpty()) {
                int value = Integer.parseInt(command);
                cpu.getIoPort(InputPort).inputToCpu(value);
            }
        }
    }
};
simulator.setDaemon(true);
simulator.start();
cpu.run(false);
}

public int acceptOutputFromCPU(int value) { // Console output to System.out
    System.out.println("\n" + value);
}
```

```

    return 0; // no interrupt needed for output
}

```

Step 4 - Here's the device driver.

```

static String driverSourceCode = ""
    JMPIMM Begin:      # Skip over data area
OutPort: EQU 10
Irq:    EQU 5
InPort: EQU 11
Buf:    0              # Inp -> Buf: -> Out
Begin:  LOADIMM $IV, Isr: # Enable interrupts
Loop:   ZEROREG $1
        ADDMEM $1, Buf:   # Add to set Flags
        JZEIMM Loop:      # Wait for Buf != 0
        INCREG $1         # Increment InputVal
        OUTREG $1, OutPort: # Echo it
        ZEROREG $2
        STORMEM $2, Buf:  # Clear the Buffer
        JMPIMM Loop:      # Else keep looping

# Here's the ISR...
Isr:    ENTER 0          # When input is rcv'd...
        CMPIMM $IN, Irq:  # Make sure it's for us
        JNZEIMM Leave:    # If not Our IRQ
        INPREG $2, InPort: # Read it
        STORMEM $2, Buf:  # Put it into Buf
Leave:   ILEAVE
Exit:
"";

```

Of course, this is a greatly oversimplified driver. We only have a one-byte queue for data and the API is simply an infinite loop waiting for input. Moreover, it does no error-checking. But it should help you to understand the fundamentals.

Normally, a device driver does not only do only input or output, but both, even for input-only or output-only devices. For instance, in order to send output to a console, the driver must first ask the console about its capabilities, resolution, and so forth. Before it sends data to it, it has to ask if it's ready to receive. So, whether your device is input or output, it usually both sends and receives.

## Connections and System Coding Areas

The connection to a device involves cabling as well I/O ports or memory-slot I/O, or direct memory access, as well as physical connections for input, output, and the generation of interrupts. Typically, this is all incorporated into an adaptor, such as a network interface card (NIC), a disk-drive interface (SATA), or a USB adaptor on a chip, which is typically mounted on the motherboard. Moreover, there are four system design areas: Driver Support, Driver API, Driver ISR, and Interface Card Firmware. Often, the peripheral manufacturer is expected to provide all of these things to support their hardware.

(1) The Driver API is simply the code that gets called by the operating system when an application wants to communicate with the device. Operating systems prevent the application program from accessing a device directly, as that endangers all apps on the computer and risks data loss or corruption. So, your code uses a File, a Socket, a Stream, or some other object to tell the operating system what it wants to do. Then the operating system calls the Driver API to do it. One aspect of

systems programming is to write Driver APIs, often in C or C++. But the Driver API cannot communicate directly with the device because, for that, we implement the driver ISR..

(2) Another aspect of systems programming is to write a driver interrupt service routine, an ISR. This is the code that actually reads and writes the data. Typically, it is invoked by an interrupt when the device is ready - when it is sending to the computer's input or when it is ready to accept output. It runs in supervisor mode (privileged or ring zero) so that it is not prohibited from accessing I/O or memory directly. The programmer has to use a bit of assembler as well as C or C++ and have a good understanding of virtual memory and other OS-specific issues. Although there are a lot of rules to follow, it is not particularly difficult to code, but it must be very fast, and it cannot contain any bugs or it will crash the computer. (Blue Screen of Death) The Driver API and the Driver ISR are bundled into one file that must be carefully installed into the operating system

(3). Optionally, there may be other (higher-level) "drivers" as well, although a systems programmer would shudder to hear us refer to them as "drivers." These are software adaptors to manage communication protocols or to present the nature of the Driver API in a manner that is expected by the operating system and code libraries. For instance, for a network, these might implement protocols for ARP, BOOTP, or DHCP. For an SSD, these "drivers" might serve to make the SSD appear to the operating system to be a spinning disk. For a USB printer, they might monitor ink supply or perform color compensation.

(4) And then there is the firmware on the interface card, often written in assembler. It is software that is burned into Flash or EPROM memory on that card. In the accompanying project (CPU) you will see that there is a device simulator, written in Java, that provides this functionality so that the CPU simulator can access the console - the keyboard and text display of your computer. The other system programs, above, are written in CPU-SIM Assembler.

*[End]*